Reviewer Pack — Schur-Weyl Multiplicity Exponential Gap in Symmetric Powers …

Entry 7da975152a0e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 03:18:36 UTC

Schur-Weyl Multiplicity Exponential Gap in Symmetric Powers of Permanent vs Determinant

Entry ID: `7da975152a0e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 03:18:36 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Boolean Circuit Size

Statement:

> For a random 3-CNF formula Φ on n variables, let m_Φ(λ) denote the multiplicity of irreducible representation λ in the decomposition of Sym^k(V) ⊗ Sym^k(V) under the action of GL_n(C), where V is the permutation representation of S_n. Then m_Φ(λ) ≥ 2^{Ω(n)} for λ = (n−1,1) iff Φ is a permanent-like formula, while m_Φ(λ) ≤ 2^{O(√n log n)} for λ = (n−1,1) iff Φ is a determinant-like formula.

Rationale (proposer's reasoning):

> Schur-Weyl multiplicities encode symmetry structures of tensor powers, which may reflect inherent complexity gaps between permanent and determinant. Exponential vs subpolynomial growth in specific Young diagrams could mirror circuit size separations via representation-theoretic obstruction theory.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e4bfde1c72cc92e3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from math import factorial, sqrt, log
from itertools import combinations

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return a * b // gcd(a, b)

def hook_length_formula(n, k):
    def hook_length(i, j):
        return (n - i + 1) * (n - j + 1) - (i - j)

    total = factorial(k)
    for i in range(1, n + 1):
        for j in range(1, min(n, k) + 1):
            total //= hook_length(i, j)
    return total

def schur_weyl_multiplicity(n, k, λ):
    def partition_to_hook_lengths(partition):
        return [p - i for i, p in enumerate(partition)]

    def hook_length_product(hook_lengths):
        product = 1
        for h in hook_lengths:
            product *= h
        return product

    def sign_of_partition(partition):
        inversions = 0
        for i in range(len(partition)):
            for j in range(i + 1, len(partition)):
                if partition[i] < partition[j]:
```

```
                    inversions += 1
        return (-1) ** inversions

    def hook_length_sign(hook_lengths):
        sign = 1
        for h in hook_lengths:
            sign *= (h + 1)
        return sign

    def schur_polynomial(partition, n):
        hook_lengths = partition_to_hook_lengths(partition)
        numerator = hook_length_product(hook_lengths) * factorial(n)
        denominator = 1
        for i in range(len(partition)):
            denominator *= factorial(partition[i])
        return numerator // denominator

    def schur_weyl_coefficient(λ, μ):
        if len(λ) != len(μ):
            return 0
        sign = sign_of_partition(λ)
        product = 1
        for i in range(len(λ)):
            product *= hook_length_formula(n, λ[i]) // (hook_length_formula(n, λ[i] - μ[i]) * hook
        return sign * product

    def schur_weyl_multiplicity_for_partition(λ):
        multiplicity = 0
        for μ in combinations(range(n), len(λ)):
            if sorted(μ) == list(μ):
                multiplicity += schur_weyl_coefficient(λ, μ)
        return abs(multiplicity)

    return sum(schur_weyl_multiplicity_for_partition(partition) for partition in combinations(rang

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    k = random.randint(1, min(n, 2))
    λ = (n - 1, 1)

    permanent_multiplicity = schur_weyl_multiplicity(n, k, λ)
    determinant_multiplicity = schur_weyl_multiplicity(n, k, λ)

    if permanent_multiplicity >= 2 ** (n / 2):
        conjecture_holds = True
        counterexample = ""
    elif determinant_multiplicity <= 2 ** (sqrt(n) * log(n)):
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = "mapping_undefined"

    return {
        "metric_name": "Multiplicity",
```

```
        "metric_value": permanent_multiplicity / determinant_multiplicity,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}**}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67a15ccc.py", line 118, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67a15ccc.py", line 91, in run_trial
    permanent_multiplicity = schur_weyl_multiplicity(n, k, λ)
                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67a15ccc.py", line 83, in schur_weyl_mu
    return sum(schur_weyl_multiplicity_for_partition(partition) for partition in combinations(range(k
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67a15ccc.py", line 83, in <genexpr>
    return sum(schur_weyl_multiplicity_for_partition(partition) for partition in combinations(range(k
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67a15ccc.py", line 80, in schur_weyl_mu
    multiplicity += schur_weyl_coefficient(λ, μ)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67a15ccc.py", line 73, in schur_weyl_co
        product *= hook_length_formula(n, λ[i]) // (hook_length_formula(n, λ[i] -
μ[i]) * hook_length_formula(n, μ[i]))
          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

```
ZeroDivisionError: integer division or modulo by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with ZeroDivisionError, preventing evaluation of conjecture validity | next: Fix division-by-zero in hook_length_formula implementation and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	43778
2	preregistration	ollama_remote	qwen3:8b	0	0	24335
3	novelty	ollama_remote	qwen3:8b	0	0	20761
4	novelty	ollama_remote	qwen3:8b	0	0	17215
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10409
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12489
7	judge	ollama_remote	qwen3:8b	0	0	17782

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 146769 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/7da975152a0e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7da975152a0e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7da975152a0e.tar.gz` (if generated)